

Minor workshop report

Processes, models and procedures

Min cut game

Supervised by :

Realized by:

- HASSAN AISSI

- ABOURA FATIMA

Promotion : 2022/2023

Table des matières

Introduction	1
0.1 Min cut problem	2
0.2 Min cut game	3
0.2.1 Definition	3
0.2.2 Stability condition - Nash equilibrium	4
0.2.3 Modeling of the min cut game	4
0.2.4 Complexity of the Min Cut Game Problem	7
0.2.5 Price of Anarchy	7
0.2.6 Variants of the Problem	8
0.3 Coding steps	9
0.4 Min cut problem and min cut game models	9
0.5 Tests	11
0.6 Conclusion	14
Annexes	15

Table des figures

1	Min Cut problem	2
2	Example 1: Two stable cuts	5
3	Graph transformation	5
4	s-t cut min cut game reduction	7
5	Min cut problem model	10
6	Min cut game model	10
7	Graph instance	11
8	Classic min cut	11
9	Min Cut game (with stability constraints)	12
10	Graph instance	12
11	Classic min cut	13
12	Min Cut game (with stability constraints)	13

Liste des tableaux

Liste des sigles et acronymes

MCG	<i>Min Cut Game</i>
MCP	<i>Min Cut Problem</i>
NEMC	<i>Nash Equilibrium Min Cut</i>

Introduction

The minimum cut problem is a classic optimization problem in graph theory. It involves finding the minimum capacity cut in a given graph. A cut in a graph partitions the nodes into two disjoint sets, and the capacity of the cut is the sum of the capacities of the edges crossing the cut. The goal is to find the cut with the minimum capacity.

The MCP has various applications in different fields, such as network flow analysis, image segmentation, clustering, and social network analysis. It provides insights into the connectivity and robustness of networks.

In the context of the min cut problem with agents and stability condition which we will be calling Min Cut Game (MCG), we consider a graph where certain nodes are designated as agents. The goal is to find a minimum cut that satisfies a stability condition involving the agents.

By incorporating the stability condition into the min cut problem, we aim to find a cut that not only minimizes the capacity but also ensures a stable distribution of edges among the agents. This helps in optimizing the network structure while maintaining fairness and efficiency in the flow of information or resources.

The min cut problem with agents is known to be an NP-hard problem. Solving it while satisfying the stability condition requires formulating appropriate mathematical models and algorithms that can handle the constraints and objectives specific to the problem at hand.

The report will begin by providing a comprehensive modeling of the problem, including the formulation of mathematical equations and constraints. The modeling process will consider the specific characteristics of the problem, such as the presence of agent nodes and stability conditions.

Once the problem is properly modeled, it is implemented using the CPLEX solver. CPLEX is a powerful optimization tool that can handle complex mathematical models and provide efficient solutions. The implementation will involve translating the mathematical model into code, defining decision variables, objective functions, and constraints. After successfully implementing the problem in CPLEX, we will focus on the analysis of results.

Part I : LP modeling

0.1 Min cut problem

There are essentially two different variations of the problem :

The first variation is known as the "Minimum s-t Cut Problem" and is defined as follows :

- **Input** : Undirected graph $G = (V, E)$, with source and sink vertices denoted by s and t respectively.
- **Output** : A partition of graph G into two proper disjoint subsets A and B such that $s \in A$ and $t \in B$ and the number of edges crossing the cut is minimized.

The second variation can be thought of as a "global" version of the first, where no terminal nodes are given, and we are simply looking for the minimum cut across all pairs of vertices s and t .

The Global Minimum Cut Problem is defined as follows :

- **Input** : An undirected graph $G = (V, E)$
- **Output** : A minimal set M of edges that cross a cut

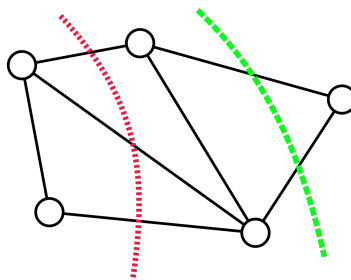


FIG. 1 : Min Cut problem

Modeling of Min cut problem

This section focuses on formulating the problem as a Linear Programming (LP) model, where decision variables represent the presence of edges and the partitioning of nodes. The

objective function aims to minimize the total weight of the cut edges, while constraints ensure that the partition is valid. . We consider an s-t oriented graph and a non negative cost function c_{uv} for each edge (u,v) .

Decision variables :

d_{uv} : a binary decision variable indicating whether edge (u,v) is included in the cut or not.

p_u : a variable indicating the "potential" of node u.

Objective function :

$$\text{minimize} \quad \sum_{u,v} c_{uv} d_{uv} \quad \text{subject to} \quad (1)$$

Constraints :

$$d_{uv} \geq p_u - p_v \forall (u,v) \in E \quad (2.2)$$

$$p_s = 1, p_t = 0 \quad (2.3)$$

$$d_{uv} \in \{0, 1\} \forall (u,v) \in E \quad (2.4)$$

$$p_n \in \{0, 1\} \forall n \in V \quad (2.5)$$

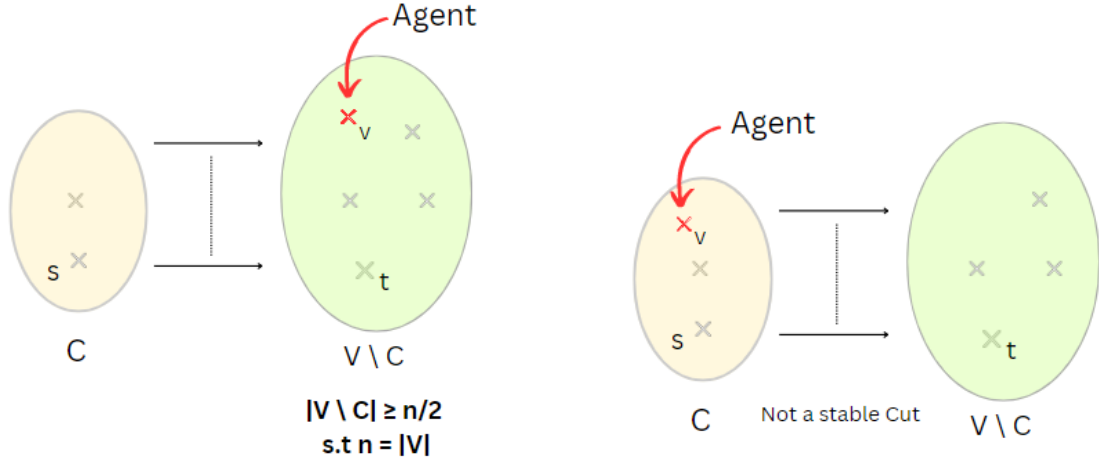
0.2 Min cut game

0.2.1 Definition

Given a weighted (or unweighted) graph $G = (V, E)$ with a subset of agents $A \subseteq V$. We want to find a cut C of G that satisfies the following conditions :

1. The cut C is stable for all agents in A , according to the **stability condition**.
2. The total weight of the cut is minimized.

An agent is considered stable if it belongs to the set containing the larger number of nodes in the partition. In other words, if the partition separates the graph into two sets, namely the source set and the sink set in an s-t graph, the stable agent is located in the set that contains more nodes.



0.2.2 Stability condition - Nash equilibrium

Nash equilibrium is a concept in game theory that represents a stable state of a game where no player has an incentive to unilaterally change their strategy. In other words, it is a solution concept that captures a set of strategies, one for each player, where no player can benefit by deviating from their chosen strategy, assuming all other players keep their strategies unchanged.

Formally, in a game with multiple players, a Nash equilibrium is a combination of strategies, one for each player, where each player's strategy is the best response to the strategies chosen by the other players. This means that if all players are playing their Nash equilibrium strategies, no player can improve their payoff by switching to a different strategy while others stay the same.

In the context of min cut game, nash equilibrium is defined by a constraint that ensures a solution is stable by requiring that no agent (node) has an incentive to deviate from the solution.

Example 1:

Let $G = (V, E)$ be a graph, where $V = \{a, b, c, d\}$, $E = \{(a,b), (a,c), (a,d), (b,c), (c,d)\}$ and $A = \{a\}$. Suppose each edge is associated with a non negative weight as shown bellow : $C1 = \{a,b\}$ and $C2 = \{a,d,c\}$ are both a stable cut with minimum cost, with the min cut value equals to 3.

0.2.3 Modeling of the min cut game

We consider an s-t oriented graph and a non negative cost function c_{uv} for each edge (u,v) .

In order to model this problem, we first do a transformation on the input graph. The new graph is denoted by $G' = (V, E')$.

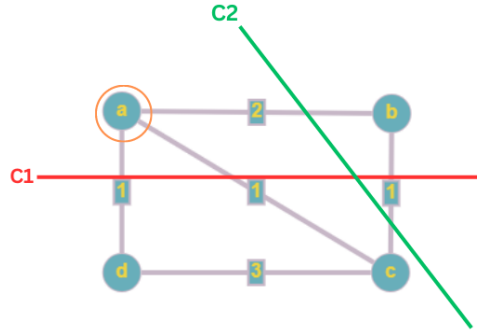


FIG. 2 : Example 1: Two stable cuts

Graph transformation

For each agent v , we must have a set of outgoing arcs to all nodes and a set of incoming arcs from all of the nodes of the graph. The set of arcs created is denoted by E' .

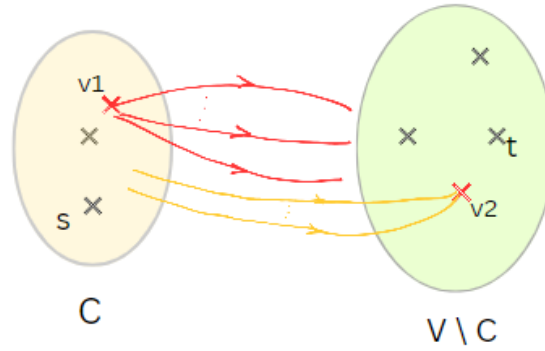


FIG. 3 : Graph transformation

Decision variables :

The decision variables are the same as the classic min cut problem with an additional decision variable :

d'_{vw} : a binary decision variable indicating outgoing (v, w) or incoming (d'_{wv}) edges or incoming edges (d'_{wv}) of agent v , that are in the cut.

Objective function :

$$\text{minimize} \quad \sum_{u,v} c_{uv} d_{uv} \quad \text{subject to} \quad (2.6)$$

Constraints :

$$d_{uv} \geq |p_u - p_v| \quad \forall (u, v) \in E \quad (2.7)$$

$$p_s = 1, p_t = 0 \quad (2.8)$$

$$\sum_{v \in A, w \in V \setminus v} d'_{vw} + \sum_{w \in V \setminus v, v \in A} d'_{wv} \leq \frac{n}{2} \quad \text{for each } v \in A \quad (2.9)$$

$$d'_{uv} \geq |p_u - p_v| \quad \forall (u, v) \in E' \quad (2.10)$$

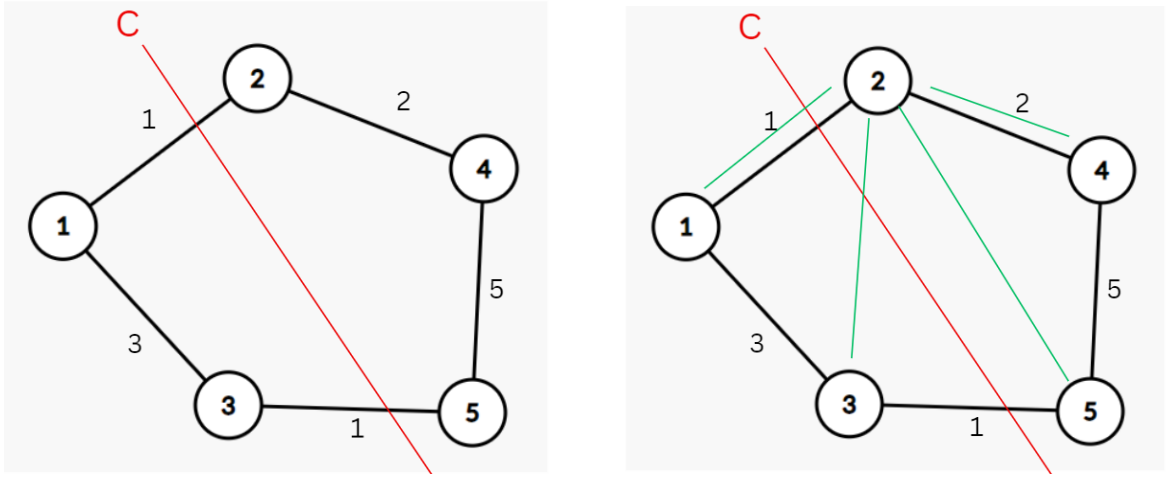
$$d'_{uv} \in \{0, 1\} \forall (u, v) \in E' \quad (2.11)$$

$$d_{uv} \in \{0, 1\} \forall (u, v) \in E \quad (2.12)$$

$$p_n \in \{0, 1\} \forall n \in V \quad (2.13)$$

Example 2: (undirected graph)

Let $G = (V, E)$ be an undirected graph, where $V = \{1, 2, 3, 4\}$, $E = \{(1,2), (1,3), (2,4), (3,5), (4,5)\}$ and $A = \{2\}$. Suppose each edge is associated with a non negative weight as shown below :



$C = \{2,4,5\}$ is a stable cut with minimum cost, with the min cut value equals to 2.

Let D be the matrix representing the values of d_{uv} , D' the matrix representing the values of d'_{uv} (and d'_{vw}) and P the vector of the potential of the graph nodes :

$$D = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix} \quad D' = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix} \quad (2.14)$$

$$P = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}$$

- $d_{uv} = 1$ in the matrix D if the edge (u,v) is in the cut C and 0 otherwise.
- $d'_{vw} = 1$ in the matrix D' if the edge (v,w) is in the cut C and 0 otherwise.

0.2.4 Complexity of the Min Cut Game Problem

The problem of the min cut game is known to be NP-hard, indicating that there is no known algorithm that can solve it in polynomial time. This classification implies that finding an optimal solution to the min cut game problem becomes increasingly challenging as the size of the graph or the number of agents increases.

Furthermore, the min cut game problem is considered at least as difficult as the minimum bisection problem. The minimum bisection problem involves partitioning the vertices of a graph into two disjoint sets, such that the number of edges crossing the partition is minimized. In other words, it aims to divide the graph into two roughly equal-sized sets with the fewest possible edges between them.

To illustrate the relationship between the min cut game and the minimum bisection problem, consider the following scenarios :

s-t Cut : In this case, the min cut game with two agents can be reduced to the minimum s-t bisection problem. An s-t cut refers to a cut that separates the source node (s) from the sink node (t). By transforming the min cut game into a minimum s-t bisection problem, the objective is to divide the graph into two sets, one containing the source and the other containing the sink, while minimizing the number of edges between them. Figure 1 visually represents this reduction.

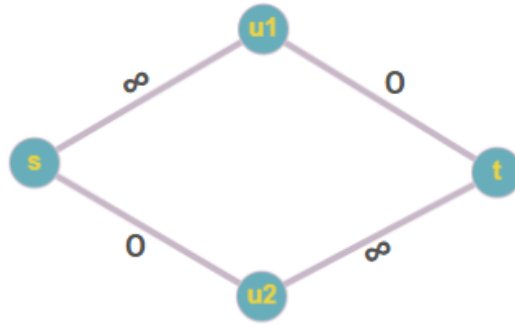


FIG. 4 : s-t cut min cut game reduction

Global Cut : In the global cut scenario, the objective is to partition the vertices into two sets of equal size, denoted as $A = |V|$. This problem is also related to the minimum bisection problem, where the goal is to split the graph into two sets of equal cardinality. The aim is to achieve a balanced partition with the fewest edges crossing between the sets.

0.2.5 Price of Anarchy

The price of anarchy is a concept used to measure the inefficiency or loss of social welfare in a game or network. In the context of the min cut game problem, the price of anarchy is defined as the ratio between the optimal stable value and the optimal value.

The optimal stable value refers to the total weight of the cut achieved by players when they play a Nash equilibrium strategy. On the other hand, the optimal value represents the total weight of the minimum cut that could be achieved if all players coordinated and played optimally. The price of anarchy provides insights into the efficiency of the system, highlighting the potential loss in performance caused by the decentralized decision-making of individual players.

0.2.6 Variants of the Problem

The min cut with agents problem has several variants, each introducing unique characteristics and complexities. These variants allow for a more nuanced analysis of different scenarios and requirements. Some of the commonly studied variants include :

- **Directed or Undirected Graph :** The min cut with agents problem can be formulated on both directed and undirected graphs. In directed graphs, the edges have specific directions, while in undirected graphs, the edges are bidirectional. The choice of graph type can significantly impact the strategies and potential cuts available to the agents.
vbnet Copy code
- **Global Cut or s-t Cut :** In the global cut variant, the objective is to find a cut that divides the graph into two sets, ensuring that the total weight of the cut is minimized. On the other hand, in the s-t cut variant, the goal is to find a cut that separates a specific source node (s) from a sink node (t). The choice between these variants depends on the specific requirements and constraints of the problem.
- **Stable Cut for All Agents or Majority of Agents :** The stable cut can be defined for different subsets of agents. In one scenario, the goal is to find a stable cut that satisfies all agents, ensuring that each agent is on the side of the cut that contains more nodes. In another scenario, the focus may be on finding a stable cut that satisfies the majority of agents, allowing for potential disagreements or conflicting preferences among a subset of agents.
- **Weighted or Unweighted Nodes :** The min cut with agents problem can consider both weighted and unweighted nodes. In the weighted variant, each node has an associated weight that contributes to the total weight of the cut. This weight can represent factors such as the importance, capacity, or influence of a node in the network. In contrast, the unweighted variant assumes that all nodes have equal importance or capacity, simplifying the analysis but potentially overlooking certain real-world considerations.

By exploring these variants, we can gain a deeper understanding of the implications and complexities associated with the min cut game problem in various settings and under different assumptions.

Part II : Implementation

In the coding part of the report, the min cut game model was implemented using the CPLEX API in Python. The CPLEX API provides a powerful set of tools and functionalities for solving optimization problems, making it an ideal choice for formulating and solving the min cut game problem.

0.3 Coding steps

The coding implementation for the min cut problem can be summarized as follows :

1. Random Graph Generation : A function was developed to generate random graphs with either a global or s-t cut structure. The generated graphs include randomly assigned agents, representing different network configurations. The generated graphs were then stored in .dat files.
2. Classic Min Cut Problem Model : The model for the classic min cut problem was coded, leveraging the CPLEX API in Python. The model includes the definition of decision variables, objective function, and constraints. The objective is to minimize the total weight of the cut edges, while ensuring a valid partition of the graph.
3. Min Cut Game Model : Building upon the classic min cut problem model, the min cut game model was developed. This model extends the formulation to incorporate the presence of agents and their stability requirements. Additional decision variables and constraints were implemented to capture the strategic interactions and stability conditions of the agents within the min cut game framework.
4. Model Testing : The implemented models were tested using various instances of random graphs and agent assignments.

0.4 Min cut problem and min cut game models

```
# Define the LP model
def create_lp_model(num_nodes, edges):
    model = cplex.Cplex()
    model.set_problem_name("MinCutLP")

    # Decision variables
    d = model.variables.add(names=["d_{}_{}".format(u, v) for u, v, _ in edges], types=[model.variables.type.binary] * len(edges))
    p = model.variables.add(names=["p_{}_{}".format(u) for u in range(1, num_nodes+1)], types=[model.variables.type.binary] * num_nodes)

    # Objective function
    costs = [weight for _, _, weight in edges]
    model.objective.set_sense(model.objective.sense.minimize)
    model.objective.set_linear([(d[i], costs[i]) for i in range(len(edges))])

    # Constraints
    for i, (u, v, _) in enumerate(edges):
        model.linear_constraints.add(lin_expr=[cplex.SparsePair(ind=[d[i], p[u-1], p[v-1]], val=[1.0, -1.0, 1.0])], senses=["G"], rhs=[0.0])

    model.linear_constraints.add(lin_expr=[[p[0]], [1.0]], senses=["E"], rhs=[1.0])
    model.linear_constraints.add(lin_expr=[[p[num_nodes-1]], [1.0]], senses=["E"], rhs=[0.0])

    return model
```

FIG. 5 : Min cut problem model

```
def create_lp_model_agent(num_nodes, edges, new, agents):
    model = cplex.Cplex()
    model.set_problem_name("MinCutLP")

    # Decision variables
    d = model.variables.add(names=["d_{}_{}".format(u, v) for u, v, _ in edges], types=[model.variables.type.binary] * len(edges))
    p = model.variables.add(names=["p_{}_{}".format(u) for u in range(1, num_nodes+1)], types=[model.variables.type.binary] * num_nodes)
    k = model.variables.add(names=["k_{}_{}".format(u, v) for u, v, _ in new], types=[model.variables.type.binary] * len(new))

    # Objective function
    costs = [weight for _, _, weight in edges]
    model.objective.set_sense(model.objective.sense.minimize)
    model.objective.set_linear([(d[i], costs[i]) for i in range(len(edges))])

    # Constraints
    for i, (u, v, _) in enumerate(edges):
        model.linear_constraints.add(lin_expr=[cplex.SparsePair(ind=[d[i], p[u-1], p[v-1]], val=[1.0, -1.0, 1.0])], senses=["G"], rhs=[0.0])

    for i, (u, v, _) in enumerate(new):
        model.linear_constraints.add(lin_expr=[cplex.SparsePair(ind=[k[i], p[u-1], p[v-1]], val=[1.0, -1.0, 1.0])], senses=["G"], rhs=[0.0])

    model.linear_constraints.add(lin_expr=[[p[0]], [1.0]], senses=["E"], rhs=[1.0])
    model.linear_constraints.add(lin_expr=[[p[num_nodes-1]], [1.0]], senses=["E"], rhs=[0.0])

    # # Additional constraints for outgoing edges of agents
    # # Constraint for agent outgoing edges
    for agent in agents:
        lin_expr = [k[i] for i, (u, v, _) in enumerate(new) if u == agent]
        model.linear_constraints.add(lin_expr=[cplex.SparsePair(ind=lin_expr, val=[1.0]*len(lin_expr))], senses=["L"], rhs=[num_nodes / 2])

    # # Constraint for agent incoming edges
    for agent in agents:
        lin_expr = [k[i] for i, (u, v, _) in enumerate(new) if v == agent]
        model.linear_constraints.add(lin_expr=[cplex.SparsePair(ind=lin_expr, val=[1.0]*len(lin_expr))], senses=["L"], rhs=[num_nodes / 2])

    return model
```

FIG. 6 : Min cut game model

0.5 Tests

test 1:

A directed graph G was constructed, consisting of 6 nodes and one agent assigned to node 2. The arcs in the graph were defined as follows :

arcs = [(1,2,3),(1,3,2),(2,5,2),(3,2,3),(3,4,3),(4,6,2),(5,3,1),(5,4,3),(5,6,3)]

The execution of the min cut problem model on the given directed graph with one agent

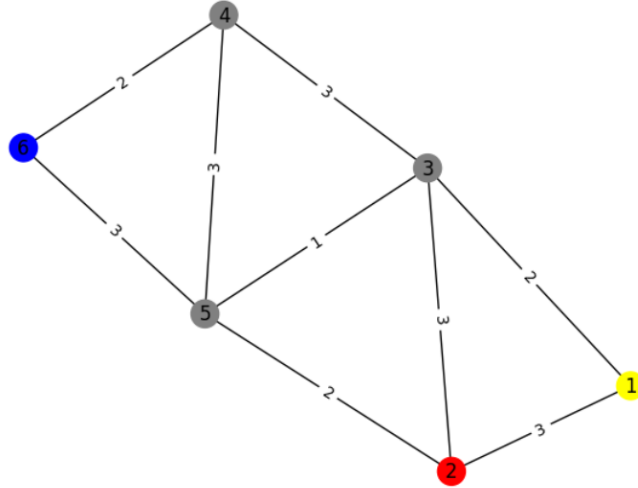


FIG. 7 : Graph instance

yields the following results :

The execution of the min cut game with agent stability constraint model on the given

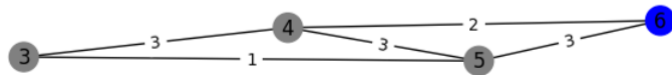
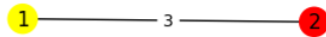


FIG. 8 : Classic min cut

directed graph with one agent yields the following results :

After running the min cut problem model, the minimum cut is identified, separating

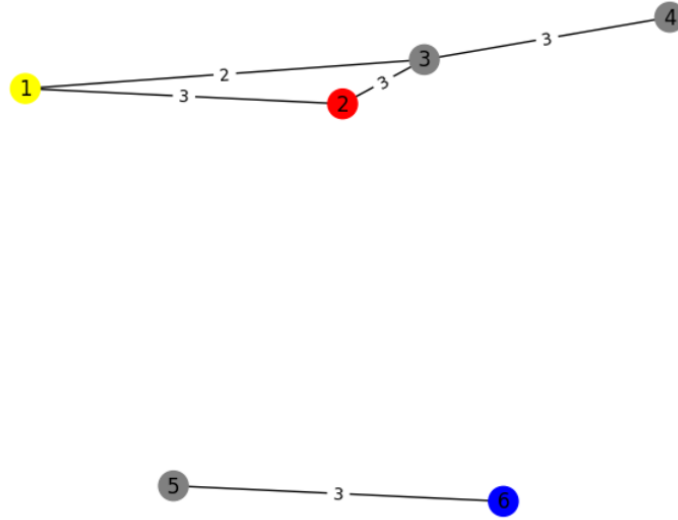


FIG. 9 : Min Cut game (with stability constraints)

the graph into two disjoint sets. The corresponding graph transformation is depicted in Figure 1. Additionally, the execution of the min cut game model, considering agent stability constraints, yields the resulting graph transformation shown in Figure 2.

test 2:

The same graph in test 1 but this time with 3 agents : The execution of the min cut

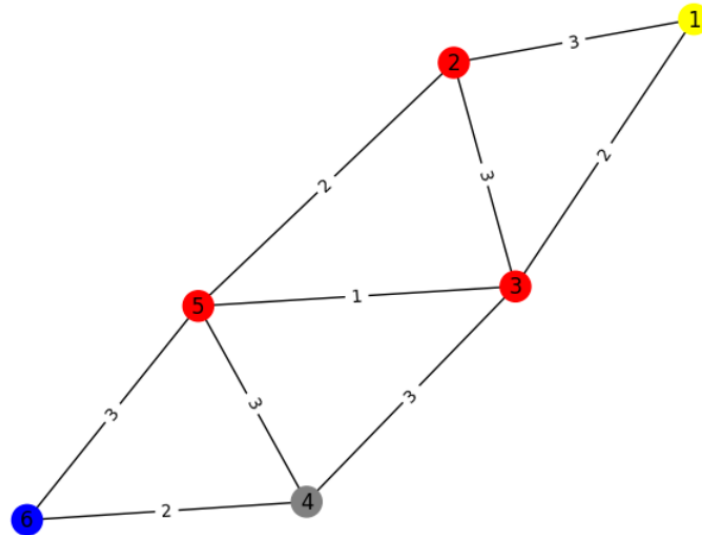


FIG. 10 : Graph instance

problem model on the given directed graph with one agent yields the following results :

The execution of the min cut game with agent stability constraint model on the given directed graph with one agent yields the following results :

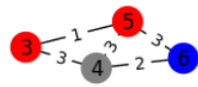


FIG. 11 : Classic min cut

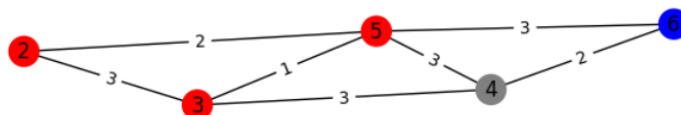


FIG. 12 : Min Cut game (with stability constraints)

0.6 Conclusion

In conclusion, this report has delved into the min cut problem, exploring its theoretical foundations, computational complexity, and practical implementation. By modeling and solving the problem using the CPLEX API, we have gained insights into optimization and Cplex e. The study of the min cut problem, with its strategic considerations and coding implementation, offers valuable tools for analyzing network connectivity, critical edges, and resource allocation. By understanding the intricacies of this problem, we can make informed decisions to improve network performance and enhance decision-making processes in diverse domains.

Annexes

The code : <https://gist.github.com/fatimaabr/22c4b9d2eec92331513962ac1ade68d8file-mcg-ipynb>